

Etapa 1 — Relatório de Modelagem do Problema

Autor

Marcus Vinícius Santos de Almeida

Disciplina

Mineração de Dados

Data de entrega

11/09/2026

Domínio escolhido

Predição de defeitos de software a partir de métricas estáticas de código

Decisão binária

O arquivo de código-fonte apresentará um defeito reportado? (SIM / NÃO)

1 Descrição do domínio

1.1 O problema de negócio

Em projetos de software de médio e grande porte, rodar a suíte de testes completa a cada alteração e submeter todo o código a revisão manual detalhada não escala: consome tempo de máquina e, principalmente, tempo de pessoas. Uma estratégia comum é **priorizar** — concentrar esforço de teste e de revisão nos trechos de código com maior probabilidade de conter defeitos.

Este trabalho modela exatamente essa priorização como um problema de classificação binária: dado um arquivo de código-fonte descrito por um conjunto de métricas estáticas e de histórico, decidir se ele deve ser tratado como propenso a defeito (SIM) ou não propenso (NÃO). A saída serve como **signal de priorização**, não como veredito — um arquivo classificado como SIM entra na frente da fila de revisão e ganha atenção extra de testes.

1.2 Por que este domínio

- **Literatura acadêmica consolidada.** Predição de defeitos a partir de métricas de código é uma sub-área madura de engenharia de software empírica, com décadas de estudos — base sólida para justificar cada feature e para a análise crítica da Etapa 4.
- **Dados fáceis de obter.** As seis features podem ser extraídas de um repositório Git com ferramentas leves e gratuitas (ou com a biblioteca padrão do Python), sem instrumentação especial nem acesso a bancos de bugs proprietários.
- **Utilidade prática real.** O resultado orienta onde gastar o orçamento limitado de teste e revisão.

1.3 O que conta como "defeito reportado"

Adota-se a convenção usual da área: um arquivo recebe rótulo SIM quando, em uma janela de observação definida (por exemplo, a versão seguinte do sistema ou os N meses seguintes), ele foi modificado por um commit de correção — associado a um item de bug em rastreador de issues, ou identificável pela mensagem ("fix", "bug", "corrigir", referência a `#issue`). Arquivos não tocados por nenhum commit de correção nessa janela recebem NÃO.

Nota

Como os dados de treinamento desta atividade são sintéticos (Seção 6 e Etapa 2), o rótulo é gerado junto com as features de forma coerente com os padrões da Seção 3.4; a definição acima documenta como o rótulo seria obtido em um cenário real.

2 Unidade de análise

A "unidade de análise" (o que o enunciado chama de *módulo*) é o **arquivo de código-fonte**. O estudo de referência para o comportamento tamanho × defeito (Koru et al., 2008) usa a **classe** como módulo. Aqui adota-se o arquivo como simplificação deliberada:

1. Nenhuma das seis features atuais depende do conceito de classe — DIT e acoplamento (CBO), que dependiam, foram removidas do modelo (Seções 3.2 e 4).
2. Arquivo é unidade universal: funciona para Python, C, SQL, qualquer linguagem, e é a granularidade nativa do Git — o que simplifica a coleta de autores e churn.
3. Evita ferramenta adicional: nível de classe exigiria um parser semântico por linguagem; nível de arquivo, as mesmas ferramentas leves bastam.

Simplificação declarada

Ao usar arquivo em vez de classe, agregamos em um só registro elementos que um estudo em nível de classe separaria (um arquivo pode conter várias classes ou funções). As métricas por registro ficam um pouco mais "grossas" e podem diluir sinais localizados numa classe específica. Assume-se esse custo em troca de simplicidade e universalidade da coleta.

3 Modelagem do problema

3.1 Rótulo alvo

O arquivo apresentará um defeito reportado na janela de observação? Valores: **SIM** (propenso a defeito) e **NÃO** (não propenso) — é a variável que o Naive Bayes estima, calculando $P(\text{SIM} \mid \text{features})$ e $P(\text{NÃO} \mid \text{features})$. Critério de rótulo: presença de commit de correção tocando o arquivo na janela (Seção 1.3).

3.2 As seis features do domínio

Nenhuma feature foi inventada ou removida nesta etapa. DIT e acoplamento (CBO) foram descartados antes por dependerem do conceito de classe e por correlacionarem fortemente com tamanho na literatura (Seção 4) — reduziriam a diversidade real de informação sem agregar sinal novo.

Feature	Coleta	Fundamentação e ponto de atenção
1. Complexidade ciclomática	radon cc	McCabe (1976) criou a métrica para apoiar o desenho de casos de teste, não para prever defeitos diretamente. Correlaciona-se fortemente com LOC (evidência empírica de relação linear estável, $R^2 \approx 0,90$ e acima); Shepperd (1988) argumenta que, para boa parte do

		software, é apenas um <i>proxy</i> para LOC. É o exemplo central de violação da hipótese de independência do Naive Bayes — discussão teórica completa na Seção 5.
2. Linhas de código (LOC)	<code>radon raw</code>	Koru et al. (2008): a relação tamanho → defeito segue lei de potência com expoente < 1 — módulos menores são <i>proporcionalmente</i> mais propensos a defeito (contraintuitivo). A discretização (3.3) precisa refletir essa não-linearidade, não assumir "mais linhas = mais risco" de forma linear.
3. Nº de autores distintos	PyDriller / <code>git log</code>	Matsumoto et al. (2010): métricas de desenvolvedor melhoram a predição. Weyuker, Ostrand & Bell (2008), "Do Too Many Cooks Spoil the Broth?": quanto mais pessoas diferentes editam um arquivo, maior tende a ser a propensão a defeito — falta de propriedade clara do código.
4. Churn relativo % das linhas do arquivo alteradas nos últimos 90 dias	PyDriller	Nagappan & Ball (2005): usar churn <i>relativo</i> ao tamanho do módulo, não o churn absoluto — o absoluto é preditor fraco e correlaciona com tamanho; a versão relativa foi desenhada precisamente para reduzir essa dependência.
5. Nº de imports dependências externas	módulo <code>ast</code> (stdlib)	Substitui DIT/CBO. Mede acoplamento de forma leve, sem depender de classe: quantos módulos/pacotes distintos o arquivo importa. Um arquivo com muitas dependências tem maior superfície para quebrar quando qualquer uma delas muda.
6. Cobertura de testes	Coverage.py	Evidência contestada. Vários estudos — inclusive um survey com 235 respostas de sete organizações (Gren & Antinyan, 2017) e um estudo empírico controlado (Inozemtseva & Holmes, 2014) — encontram correlação nula ou fraca entre cobertura e (ausência de) defeitos. Não tratar "mais cobertura → menos defeito" como relação óbvia; é limitação assumida do modelo, retomada na Etapa 4.

3.3 Discretização de cada feature

Os pontos de corte são razoáveis para código Python de projeto real; o que precisa se manter é a lógica das faixas, não a calibração fina de cada número.

Feature	Baixo	Médio	Alto	Observação
Complexidade ciclomática	≤ 10	11–20	> 20	Bandas clássicas de McCabe. Em código real "alta" concentra risco mas também parte de sinal redundante com LOC (Seção 5); nos dados sintéticos deste projeto essa redundância foi deliberadamente removida (Etapa 2).
LOC	< 50	50–200	> 200	Curto $\neq$ seguro (Koru et al.). Relação em U: <i>médio</i> tende a ser mais tranquilo; curto e longo carregam risco por motivos diferentes.

Nº de autores	1	2–3	≥ 4	"Muitos cozinheiros" dilui responsabilidade; um autor indica propriedade clara.
Churn relativo	< 10%	10–30%	> 30%	Relativo ao tamanho do arquivo (Nagappan & Ball). Alto = área instável.
Nº de imports	0–3	4–8	> 8	Muitos = maior superfície de dependência externa.
Cobertura de testes	< 50%	50–80%	> 80%	Feature de sinal fraco/contestado, incluída para análise crítica.

Notas sobre a discretização

- LOC e complexidade não são discretizadas assumindo relação linear simples com risco — para LOC isso é explícito no formato em U (Koru et al., 2008).
- Churn é sempre relativo ao tamanho, nunca contagem absoluta.
- Todas as features têm exatamente três categorias — mantém as tabelas de verossimilhança pequenas e explicáveis linha a linha, e reduz células com contagem zero.
- Cada corte é um número que o autor consegue justificar oralmente; nenhum é fruto de otimização automática.

3.4 Lógica intuitiva dos padrões de risco

Sinais que puxam para SIM (mais propenso a defeito):

- **Código muito ramificado** (complexidade alta): mais caminhos de execução, mais condições, mais lugares onde a lógica pode estar errada, mais difícil de cobrir com testes.
- **Muitas mãos no mesmo arquivo** (autores muitos): sem "dono", convenções se misturam e suposições de um autor são quebradas por outro.
- **Arquivo que muda o tempo todo para o tamanho que tem** (churn alto): indica requisito instável ou design que ainda não assentou.
- **Arquivo pequeno, porém denso** (LOC curto + complexidade média/alta): o caso contraintuitivo de Koru et al. — lógica comprimida em pouco espaço, frequentemente utilitários centrais chamados de todo lado. **Na modelagem deste projeto, esse padrão aparece como a soma dos dois efeitos individuais** (LOC baixo + complexidade alta), não como um bônus de interação plantado — ver Seção 5 e a Etapa 4.
- **Muitas dependências externas** (imports muitos): amplia a superfície de quebra por mudanças em bibliotecas importadas.
- Cobertura baixa contribui *fracamente* para SIM (feature contestada).

Sinais que puxam para NÃO (menos propenso a defeito):

- Complexidade baixa + LOC médio + churn baixo + autores um/poucos: perfil estável, simples, com dono claro e pouca rotatividade.
- Cobertura alta contribui *fracamente* para NÃO.

Combinações ambíguas

Arquivo longo e complexo, mas com churn baixo, autor único e cobertura alta — grande e intrincado, porém maduro e bem cuidado. É justamente nesses perfis mistos que, num cenário real onde as features estivessem correlacionadas (Seção 5), a independência ingênua entre elas mais pesaria — os casos de teste da Etapa 4 exploram esse tipo de perfil.

4 Fundamentação acadêmica

Referências em formato ABNT. Cada entrada destaca o ponto de atenção que traz para a modelagem.

McCABE, T. J. A Complexity Measure. *IEEE Transactions on Software Engineering*, v. SE-2, n. 4, p. 308–320, dez. 1976. Define a complexidade ciclomática a partir do grafo de fluxo de controle para apoiar o desenho e dimensionamento de casos de teste — não prever defeitos. Usar a métrica como preditor é extensão posterior da comunidade.

SHEPPERD, M. A Critique of Cyclomatic Complexity as a Software Metric. *Software Engineering Journal*, v. 3, n. 2, p. 30–36, mar. 1988. Mostra que a complexidade ciclomática tem fundamentação teórica frágil e que, para uma classe ampla de software, não supera a simples contagem de linhas. Base para tratar complexidade × LOC como o par mais problemático do modelo **em código real** (Seção 5).

Evidência empírica complexidade × LOC. Estudos posteriores medem relação linear estável entre as duas métricas, com R^2 frequentemente acima de 0,90 (chegando a $\approx 0,93$ em amostras grandes) — quase a mesma informação estatística **em código real, não nos dados sintéticos deste projeto** (Seção 5).

KORU, A. G.; EL EMAM, K.; ZHANG, D.; LIU, H.; MATHEW, D. Theory of Relative Defect Proneness. *Empirical Software Engineering*, v. 13, n. 5, p. 473–498, out. 2008. Tamanho (LOC) e propensão a defeito seguem lei de potência com expoente < 1 — módulos menores são proporcionalmente mais propensos a defeito.

Nota metodológica citável

Os autores evitaram calcular "densidade de defeitos" ($\text{bugs} \div \text{LOC}$) porque essa divisão gera correlação negativa artificial com o tamanho; modelaram a probabilidade de defeito diretamente. Nosso rótulo é binário, então não corremos esse risco — evidência de rigor a citar na Etapa 4.

NAGAPPAN, N.; BALL, T. Use of Relative Code Churn Measures to Predict System Defect Density. In: *27th International Conference on Software Engineering (ICSE)*, 2005, St. Louis. p. 284–292. Churn relativo prediz densidade de defeito muito melhor que o absoluto, e foi desenhado para reduzir a dependência de tamanho.

MATSUMOTO, S. et al. An Analysis of Developer Metrics for Fault Prediction. In: *6th PROMISE*, 2010, Timișoara. Art. 18. Métricas de desenvolvedor agregam poder preditivo além das métricas de produto.

WEYUKER, E. J.; OSTRAND, T. J.; BELL, R. M. Do Too Many Cooks Spoil the Broth? *Empirical Software Engineering*, v. 13, n. 5, p. 539–559, out. 2008. Efeito direto do número de desenvolvedores sobre a propensão a defeito.

INOZEMTSEVA, L.; HOLMES, R. Coverage Is Not Strongly Correlated with Test Suite Effectiveness. In: *36th ICSE*, 2014, Hyderabad. p. 435–445. Controlando o tamanho da suíte, cobertura tem correlação fraca com a capacidade de detectar defeitos.

GREN, L.; ANTINYAN, V. On the Relation Between Unit Testing and Code Quality. In: *43rd SEAA*, 2017, Viena. p. 52–56. Survey com 235 respostas de 7 organizações — sem correlação relevante entre

cobertura de testes unitários e defeitos pós-teste.

Síntese para a Etapa 4

- Em código real, complexidade e LOC compartilham quase toda a informação ($R^2 \approx 0,93$) — a principal violação de independência que a literatura documenta para este domínio (Seção 5).
Nos dados sintéticos deste projeto essa correlação foi deliberadamente removida.
- Tamanho → defeito é não-linear (lei de potência) — "pequeno" não é sinônimo de "seguro". Essa relação é modelada por feature isolada (LOC) e continua presente nos dados sintéticos, pois não depende de correlação com outra feature.
- Cobertura de testes: relação com defeitos é contestada; limitação assumida.

5 Violações de independência do Naive Bayes: o que a literatura documenta

O Naive Bayes assume que as features são condicionalmente independentes dado o rótulo. **Esta seção é uma discussão teórica, não uma antecipação de achado empírico nos dados deste projeto:** descreve o que a literatura de engenharia de software empírica documenta sobre a relação entre estas métricas em código real.

Par de features	O que a literatura documenta	Referência
Complexidade ciclomática × LOC	Correlação empírica forte e bem estabelecida (R^2 frequentemente acima de 0,90, $\approx 0,93$ em amostras grandes) — as duas métricas carregam quase a mesma informação estatística em código real.	Shepperd (1988); evidência empírica posterior (Seção 4)

Este é o par mais estudado e mais citado como violação de independência nesse domínio, e é por isso que recebe destaque nesta seção. Não há, na literatura consultada, evidência igualmente forte e direta de correlação entre pares para as demais quatro features (autores, churn, imports, cobertura) — cada uma tem fundamentação própria (Seção 3.2), mas não uma métrica de correlação cruzada tão documentada quanto complexidade × LOC.

Importante — isto não é uma previsão sobre os dados deste projeto

A massa de treinamento sintética da Etapa 2 foi construída, por decisão de projeto ("Naive Bayes puro", ver CLAUDE.md), com as 6 features independentes entre si: nenhuma feature do gerador lê o valor de outra. A matriz de correlação de Pearson calculada sobre esses dados (Etapa 2, §2.2) confirma isso — todos os pares saem com $|r|$ muito próximo de 0, incluindo complexidade × LOC. Ou seja: a violação de independência descrita acima é real na literatura sobre código real, mas **não existe nos nossos dados** — não porque o modelo "escapou" dela, mas porque os dados foram desenhados de propósito para não a conter, isolando assim o comportamento teórico do algoritmo no cenário em que sua premissa central é verdadeira.

A consequência prática dessa escolha — o que ela valida e o que ela deixa de capturar sobre um cenário real de implantação — é o tema central da Reflexão Crítica da Etapa 4, e não é antecipada aqui para não adiantar conteúdo de

etapa futura.

6 Nota metodológica sobre o uso de IA

O domínio, o conjunto de seis features, os critérios de discretização e as referências deste relatório foram construídos e refinados em diálogo com uma IA generativa, usada como parceira de modelagem. O processo não foi de aceitação passiva: cada alegação teórica (a lei de potência de Koru et al., a correlação $R^2 \approx 0,93$, a fragilidade da relação cobertura $\leftrightarrow$ defeitos, a recomendação de churn relativo) foi submetida a checagem cruzada contra a literatura original antes de ser incorporada. Features cogitadas inicialmente (DIT, CBO, Halstead) foram descartadas após essa verificação mostrar forte correlação com tamanho. Todo o conteúdo é passível de defesa oral, linha a linha, pelo autor.

A Perguntas de defesa (referência rápida)

Por que arquivo e não classe como módulo?	Nenhuma das seis features depende do conceito de classe (DIT/CBO removidas por isso); arquivo é unidade universal, granularidade nativa do Git, evita parser semântico por linguagem.
Por que só 6 features?	Atende ao mínimo da atividade (6 a 8). Cada feature adicional testada (Halstead, CBO) mostrou forte correlação com LOC — reduziria diversidade sem agregar sinal novo.
Por que os dados são sintéticos?	Exigência explícita da Etapa 2 do enunciado — não é limitação técnica.
Por que as features foram geradas independentes, se a literatura diz que complexidade e LOC se correlacionam no mundo real?	Decisão deliberada de simulação ("Naive Bayes puro"): testar o Naive Bayes no cenário em que sua própria suposição central é verdadeira, isolando o comportamento teórico do algoritmo sem misturar com o efeito de uma violação plantada. Declarado explicitamente como limitação de realismo da simulação — ver Reflexão Crítica da Etapa 4.
Alguma feature fraca?	Cobertura de testes — evidência contestada (Inozemtseva & Holmes, 2014; Gren & Antinyan, 2017). Limitação assumida, não ignorada.
Se não há violação nos dados, qual é a "crítica" da Etapa 4?	Muda de "o modelo erra porque os dados violam a suposição" para "testamos o modelo no cenário ideal onde a suposição é verdadeira; ele se comporta como a teoria prevê, mas isso não reflete um cenário real de implantação". A limitação é sobre o realismo da simulação, não sobre um erro do modelo.

B Prompts que moldaram a modelagem

Documentação do diálogo exigida pelo enunciado — resumo dos prompts/perguntas-chave e o que, após verificação, foi incorporado.

#	Pergunta à IA (resumo)	O que foi incorporado após verificação
1	Sugira um domínio de decisão binária com literatura sólida, dados fáceis de obter e utilidade prática real.	Predição de defeitos a partir de métricas estáticas, pela combinação dos três critérios.
2	Quais métricas de código são preditores clássicos de defeito? Com referência primária.	Lista inicial (LOC, complexidade, Halstead, DIT, CBO, churn, dev. metrics, cobertura); cada referência conferida antes de seguir.
3	Complexidade ciclomática é redundante com LOC? Existe medida empírica?	Confirmação via Shepperd (1988) e $R^2 \approx 0,93$. Decisão de manter as duas como exemplo central de violação de independência, discutido teoricamente na Seção 5.

4	Módulo maior é sempre mais propenso a defeito? Forma funcional da relação?	Koru et al. (2008): lei de potência. Decisão de discretizar LOC em risco não-linear (U).
5	Churn absoluto ou relativo? Qual prediz melhor?	Nagappan & Ball (2005): relativo ao tamanho. Feature 4 redefinida como razão.
6	Métrica de acoplamento que não dependa de classe — imports serve? Risco de virar proxy de tamanho?	Adoção de nº de imports via <code>ast</code> ; DIT/CBO removidos.
7	Cobertura de testes prediz bem ausência de defeitos? Evidência contrária?	Inozemtseva & Holmes (2014); Gren & Antinyan (2017). Feature mantida como limitação assumida.
8	Proponha cortes baixo/médio/alto coerentes com Python real e as ressalvas acima.	Faixas da Seção 3.3, marcadas como recalibráveis na Etapa 2.
9	Revise as referências em ABNT: autor, ano, veículo, o que cada estudo afirma.	Conferência na fonte primária e ajuste das paráfrases ao que os textos sustentam.
10	Ao gerar os dados sintéticos da Etapa 2, devemos plantar as correlações que a literatura documenta (ex.: complexidade × LOC) ou gerar as 6 features independentes entre si?	Decisão de projeto "Naive Bayes puro" , registrada no CLAUDE.md: gerar as 6 features independentes entre si, deixando só o rótulo dependente das features. Testa o classificador no cenário em que sua própria suposição de independência é verdadeira por construção; a correlação real documentada na literatura passa a ser tratada como limitação de realismo da simulação (Seção 5 e Reflexão Crítica da Etapa 4), não como achado a "descobrir" nos dados. Levou à reescrita do gerador (Etapa 2), da Seção 5 deste relatório e da análise da Etapa 4.

Fim do Relatório de Modelagem — Etapa 1.